

AI 데이터 분석 · 8일차

while · 흐름 제어 · 리스트 데이터 처리

멈출 때를 스스로 정하고, 여러 값을 골라 담는다

while · break · continue · 플래그 · for+if 필터링 · 새 리스트 · 튜플

교안 05_02(while) · 05_03(리스트 처리) · 06_01(튜플 도입) 정리

어제의 for·if에 "멈춤"과 "모으기"가 더해지는 날

2026-07-30 AI 데이터 분석 교육 정리 (8일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

어제 배운 `for`는 "몇 번 돌지"를 미리 정해야 했습니다. 오늘 배우는 `while`은 **횟수를 몰라도** 조건이 만족될 때까지 돌고, `break`·`continue`는 반복 도중에 흐름을 바꿉니다. 후반부에서는 그 반복으로 리스트를 걸러 새 리스트를 만드는 실전 패턴을 익힙니다.

반복문 `while` **키워드** `break`·`continue` **패턴** 플래그·필터링 **리스트** 중첩·행열 **튜플** `(,)`

연결

6일차 리스트 + 7일차 `for`·`if`가 오늘 하나로 합쳐집니다. 빈 리스트 → `for` → `if` → `append` 이 네 줄이 오늘의 결론이자, 앞으로 `pandas`로 하게 될 데이터 필터링의 원형입니다.

목차

	장 / STEP	무엇을 배우나
CHAPTER 0	오늘의 지도 — <code>for</code> 와 <code>while</code>	횟수를 아는가, 모르는가
STEP 1	<code>while</code> 문의 기본 구조	조건 · 갱신 · 무한 루프 방지
STEP 2	종료 조건 설계하기	시작·종료·갱신 3단계 · <code>for</code> 와 선택 기준
STEP 3	<code>break</code> 와 <code>continue</code>	반복 중단 · 회차 건너뛰기 · <code>while True</code>
STEP 4	플래그와 검색 패턴	<code>found</code> 변수 · 최댓값 찾기
STEP 5	리스트 데이터 처리 — 필터링	<code>for</code> + <code>if</code> · 조건에 맞는 값만
STEP 6	새 리스트 만들기	빈 리스트 + <code>append</code> · 가공 · 부분 평균
STEP 7	리스트 안의 리스트	행과 열 · 대괄호 두 번
STEP 8	튜플 — 묶어서 보호하기	소괄호 · 쉼표 규칙 · 불변성

오늘의 지도 — for와 while

횟수를 아는가, 모르는가

반복문이 두 개인 이유는 상황이 다르기 때문입니다. "5번 출력"처럼 횟수가 정해져 있으면 `for`, "정답 맞힐 때까지"처럼 몇 번 돌지 모르면 `while`입니다.

상황	반복문	예시
횟수 · 범위가 정해짐	<code>for</code>	5번 출력 · 리스트 N개 처리
상태로 멈춤	<code>while</code>	정답 맞힐 때까지
조건 만족까지	<code>while</code>	목표값 도달까지 · 합이 100 넘을 때까지

개념

결정적 차이 하나: `for`는 알아서 멈추지만 `while`은 내가 멈추게 해야 한다는 것입니다. `for i in range(5)`는 5번 돌고 자동으로 끝나지만, `while`은 조건이 거짓이 되도록 내가 값을 바꿔주지 않으면 영원히 돌립니다. 그래서 `while`을 쓸 땐 "어떻게 멈출까"를 먼저 설계합니다.

오늘의 흐름



STEP 1

while 문의 기본 구조

조건이 거짓이 될 때까지

반복문 while 조건이 참인 동안 반복

정의 while 조건식: 아래 들여쓴 코드를 조건이 참인 동안 계속 반복합니다. 조건이 거짓이 되면 멈춥니다.

형태 while 조건식: 반복할 코드 (조건을 거짓으로 만드는 갱신)

```
count = 1
while count <= 3:
    print(count)
    count = count + 1    # 이 줄이 없으면 안 멈춤!
# 출력: 1, 2, 3
```

```
▶ 1
▶ 2
▶ 3
```

왜 쓰나 for는 range가 끝나면 자동으로 멈추지만, while은 조건이 거짓이 되어야만 멈춥니다. 그래서 반복 안에서 조건 변수를 반드시 바꿔줘야 합니다.

자주 하는 실수

count += 1을 빼먹으면 조건이 영원히 참이라 무한 루프에 빠집니다. 실행이 안 멈추면 터미널에서 **Ctrl + C**로 강제 중단하세요.

개념 무한 루프를 막는 3점검 while 쓰기 전 체크리스트

정의 시작값·종료 조건·갱신 — 이 셋 중 하나라도 빠지면 무한 루프이거나 오작동입니다.

count = 1	# ① 시작값 — 반복 전에 변수가 있는가
while count <= 3:	# ② 종료 조건 — 언젠가 거짓이 되는가
print(count)	
count += 1	# ③ 갱신 — 거짓 방향으로 값이 변하는가

왜 쓰나 수업 노트의 체크리스트가 정확히 이 셋입니다. while을 쓸 때마다 세 줄을 손가락으로 짚어가며 확인하는 습관을 들이세요.

응용 ③번 갱신을 반복 안에서 되돌려 버리는 실수가 특히 잦습니다. 아래는 값을 매번 0으로 재할당해 영원히 멈추지 않는 예입니다.

```
count = 1
while count >= 1:
    count = 0      # x 매 회차 0으로 되돌림 → 다시 += 1 해도 1
    print(count)
    count += 1     # 조건 count >= 1 이 계속 참 → 무한 루프
```

▶ 1 1 1 1 ... (Ctrl+C로 중단해야 함)

주의

7일차의 "누적 변수 초기화를 반복 안에 두면 안 된다"와 같은 실수 유형입니다. **반복 안에서 값을 되돌리고 있지 않은가**를 늘 확인하세요.

점검 항목	확인 내용	빠뜨리면
시작값	변수가 반복 전에 정해져 있는가	오작동 (NameError)
종료 조건	언젠가 거짓이 되는가	무한 루프
갱신	거짓 방향으로 값이 변하는가	무한 루프

STEP 2

종료 조건 설계하기

멈추는 법을 먼저 정한다

while은 멈추는 법을 직접 설계해야 안전합니다. 막막할 땐 세 단계로 쪼개서 생각하세요.



패턴 목표값 도달까지 반복하기 실습 1 — 숫자 맞추기

정의 "정답을 맞힐 때까지"처럼 횟수를 알 수 없는 반복의 대표 사례입니다.

```
answer = 7          # ① 목표값
guess = 0           # ① 시작 상태 (아직 정답 아님)
while guess != answer: # ② 종료 조건: "아직 정답이 아니다"
    guess = int(input("맞혀 보세요: ")) # ③ 갱신
print("정답입니다!")
```

▶ 맞혀 보세요: 3 → 계속

▶ 맞혀 보세요: 7 → 정답입니다!

왜 쓰나 종료 조건은 "멈출 상태의 반대"로 씁니다. 멈추고 싶은 건 "정답일 때"이므로 조건은 `guess != answer`(정답이 아닌 동안)입니다.

응용 같은 뼈대로 up·down 게임을 만들 수 있습니다. 조건 분기를 안에 넣으면 힌트까지 주는 프로그램이 됩니다.

```
import random
num = random.randint(1, 50)
found = False
while not found: # not found = 아직 못 맞힘
    answer = int(input("정수 입력: "))
    if answer > num:
        print("down!")
    elif answer < num:
        print("up!")
    else:
        found = True
print("정답입니다! 게임 종료")
```

▶ down! / up! / ... / 정답입니다! 게임 종료

주의

`while found != True`보다 `while not found`가 더 파이썬다운 표현입니다. 불리언 변수는 `== True`를 붙이지 않고 그대로 조건에 씁니다.

개념 시작값을 "절대 안 걸리는 값"으로 첫 회차를 반드시 돌게

정의 `while`은 조건을 **먼저** 검사하므로, 시작값이 이미 조건을 만족하면 한 번도 실행되지 않습니다.

```
# x 시작값이 정답과 같으면 반복이 아예 안 됨
answer = 7
guess = 7
while guess != answer:      # 처음부터 거짓 → 0회 실행
    guess = int(input())

# ✓ 절대 정답이 될 수 없는 값으로 시작
guess = 0                   # 또는 -1
```

왜 쓰나 그래서 실습에서 `guess = 0`이나 `a = -1`처럼 **입력 범위 밖의 값**을 시작값으로 둡니다. 1~10을 맞추는 게임이면 0이나 -1이 안전하죠.

STEP 3

break 와 continue

반복 도중에 흐름 바꾸기

키워드 break 반복 전체를 즉시 종료

정의 break를 만나면 남은 회차와 상관없이 반복문을 즉시 빠져나갑니다.

```
for i in range(1, 11):
    if i == 5:
        break          # 5를 만나면 즉시 종료
    print(i)
# 출력: 1, 2, 3, 4
```

```
▶ 1
▶ 2
▶ 3
▶ 4
```

왜 쓰나 "찾았으니 그만" 상황에 씁니다. 10,000개를 훑다가 3번째에서 찾으면 나머지 9,997번은 돌 필요가 없죠 — 속도에 직접 영향을 줍니다.

응용 while True와 짝을 이루면 "일단 무한히 돌되, 특정 신호가 오면 나간다"는 구조가 됩니다. 사용자 입력 처리의 표준 형태입니다.

```
while True:
    # 조건만 보면 무한 반복
    x = input("입력 (q=종료): ")
    if x == "q":
        # 종료 신호
        break          # 여기서 빠져나감
    print("입력받은 값:", x)
print("반복을 나오면 이후 코드가 실행됨")
```

▶ 입력받은 값: 5 / 입력받은 값: 7 / (q 입력) → 반복 종료

설비 적용

설비 응용: 측정값을 누적하다 합계가 기준을 넘으면 즉시 중단 — `if input_sum > 15: break`. 수업에서 만든 누적 입력 프로그램이 이 형태입니다.

키워드 continue 이번 회차만 건너뛰기

정의 continue는 이번 회차의 남은 코드를 건너뛰고 **다음 회차로 넘어갑니다**. 반복 자체는 계속됩니다.

```
for i in range(1, 7):
    if i % 2 == 0:
        continue    # 짝수면 아래를 건너뛴
    print(i)
# 출력: 1, 3, 5
```

▶ 1
▶ 3
▶ 5

왜 쓰나 "이건 거르고 계속"에 씁니다. 다만 위 코드는 `if i % 2 == 1: print(i)`로도 같은 결과라, continue는 건너뛴 조건이 복잡하거나 아래 코드가 길 때 진가를 발휘합니다.

구분	break	continue
하는 일	반복 전체 종료	이번 회차만 건너뛴
이후 흐름	반복 바깥으로 나감	다음 회차로 계속
쓰는 목적	찾았으니 그만	거르고 계속

주의

break는 **자기가 속한 반복문 하나만** 빠져나옵니다. 중첩 반복에서 안쪽 break는 안쪽 for만 끝내고 바깥 for는 계속 돛니다 — 둘 다 나가려면 플래그(STEP 4)를 함께 씁니다.

STEP 4

플래그와 검색 패턴

찾았는지 여부를 기억하는 변수

패턴 플래그 변수 True/False로 상태 기록

정의 "찾았다/못 찾았다" 같은 상태를 담은 불리언 변수입니다. 반복 밖에서 False로 시작해, 조건을 만나면 True로 바꿉니다.

```
found = False          # 아직 못 찾음
for v in [70, 92, 65]:
    if v > 80:
        found = True    # 찾았다고 기록
        break           # 더 볼 필요 없음

if found:
    print("이상값 있음")
else:
    print("모두 정상")
```

▶ 이상값 있음

왜 쓰나 반복이 끝난 뒤에 "그래서 찾았나?"를 판단하려면 상태를 어딘가 남겨야 합니다. 반복 안에서만 출력하면 "하나도 못 찾았을 때"를 처리할 수 없거든요.

응용 break와 함께 쓰면 "발견 즉시 중단"이라 빠릅니다. 반대로 개수까지 세려면 break 없이 끝까지 둡니다.

```
# 존재 여부만 → break로 즉시 중단 (빠름)
found = False
for v in temps:
    if v > 80:
        found = True; break

# 개수까지 → break 없이 끝까지
count = 0
for v in temps:
    if v > 80:
        count += 1
```

주의

if found:는 if found == True:와 같습니다. 불리언 변수는 비교 없이 그대로 조건에 쓰는 것이 관례예요.

패턴 최댓값 찾기 첫 값을 기준으로 갱신

정의 첫 값을 잠정 최댓값으로 두고, 더 큰 값을 만날 때마다 갱신합니다.

```
first = int(input("1번째 입력값: "))
max_value = first          # 비교 대상이 없으니 첫 값이 최댓값

for i in range(4):
    v = int(input(f"{i + 2}번째 입력: "))
    if v > max_value:      # 더 크면
        max_value = v      # 갱신
print("최댓값:", max_value)
```

▶ 입력 25, 32, 28, 35, 27 → 최댓값: 35

왜 쓰나 "현재까지의 최고 기록"을 변수 하나에 계속 덮어쓰는 방식입니다. 리스트가 아무리 길어도 변수 하나면 되죠.

응용 리스트가 이미 있다면 첫 원소로 시작하는 형태가 더 깔끔합니다. 파이썬 내장 `max()`와 결과가 같은지 대조해 보세요.

```
temps = [25, 32, 28, 35, 27]
biggest = temps[0]          # 첫 값으로 시작
for t in temps:
    if t > biggest:
        biggest = t
print(biggest)              # 35
print(max(temps))          # 35 (내장 함수와 동일)
```

▶ 35
▶ 35

개념

`f"{i + 2}번째"`에서 +2를 하는 이유 — 1번째는 이미 받았고 `i`는 0부터 시작하므로, 화면에 보이는 순번을 맞추려면 2를 더합니다.

STEP 5

리스트 데이터 처리 — 필터링

for 안에 if를 넣는다

리스트는 담아 두는 것만으로 끝이 아니라, 값을 하나씩 꺼내 처리해야 의미가 생깁니다. 값 100개를 인덱스로 출력하면 print를 100줄 써야 하지만, 반복문이면 3줄입니다.

패턴 for + if — 조건에 맞는 값만 오늘의 핵심 조합

정의 for로 값을 하나씩 꺼내고, if로 검사해 조건이 참인 값만 처리합니다.

```
temps = [25, 32, 28, 35, 27]
for t in temps:
    if t > 30:
        print(t, "고온 주의")
```

- ▶ 32 고온 주의
- ▶ 35 고온 주의

왜 쓰나 들여쓰기 단계가 실행 영역을 결정합니다 — 0단계는 한 번, 1단계(for 안)는 매 바퀴, 2단계(if 안)는 조건이 참일 때만.

응용 elif를 더하면 여러 구간으로 나눠 분류할 수 있습니다.

```
temps = [25, 32, 28, 35, 19]
for t in temps:
    if t >= 30:
        print(t, "고온")
    elif t < 20:
        print(t, "저온")
    # 20~29는 아무것도 안 함 (정상)
```

- ▶ 32 고온
- ▶ 35 고온
- ▶ 19 저온

패턴 조건 + 누적 — 부분 평균 실습 3

정의 조건에 맞는 값만 합계·개수에 더해 그 값들만의 평균을 냅니다.

```

temps = [25, 32, 28, 36, 27, 31, 24]
total = 0
count = 0
for t in temps:
    if t > 30:          # 조건에 맞는 값만
        total += t
        count += 1
print("고온 평균:", total / count)

```

▶ 고온 평균: 33.0 (32+36+31=99, 3개)

왜 쓰나 전체 개수가 아니라 **조건에 맞는 개수**로 나눠야 부분 평균이 됩니다. `len(temps)`로 나누면 틀린 값이 나오죠.

자주 하는 실수

조건에 맞는 값이 하나도 없으면 `count`가 0이라 **ZeroDivisionError**가 납니다. 나누기 전에 `if count > 0:`으로 확인하세요.

실습 2 — 두 조건을 모두 만족하는 값

```

hours = [3, 8, 12, 6, 10, 4, 9]
for h in hours:
    if h >= 5 and h <= 10:      # 또는 if 5 <= h <= 10:
        print(h)

```

▶ 8
▶ 6
▶ 10
▶ 9

주의

경계값 확인이 핵심입니다. 정확히 5와 10이 포함되는지 꼭 테스트하세요. `>`를 썼다면 5와 10이 빠집니다(7일차 경계 표현 참고).

STEP 6

새 리스트 만들기

빈 그릇 + append

패턴 빈 리스트에서 시작해 채우기 가공 패턴

정의 반복 **밖에서** 빈 리스트를 만들고, 반복 안에서 append로 결과를 모읍니다.

```
temps = [25, 26, 24, 28]
doubled = []           # 빈 그릇을 밖에서
for t in temps:
    doubled.append(t * 2) # 가공해서 담기
print(doubled)
```

▶ [50, 52, 48, 56]

왜 쓰나 원본은 그대로 두고 **가공한 새 리스트**를 얻습니다. 6일차의 "원본 보존"과 같은 사고방식이에요.

응용 실습 5의 화씨 변환이 이 패턴입니다. 계산식만 바꾸면 어떤 가공도 가능합니다.

```
celsius = [25, 26, 24, 28, 27]
fahrenheit = []
for c in celsius:
    fahrenheit.append(c * 1.8 + 32)
print(fahrenheit)
```

▶ [77.0, 78.8, 75.2, 82.4, 80.6]

자주 하는 실수

빈 리스트를 반복 **안**에 두면 매 회차 새로 만들어져 마지막 값 하나만 남습니다. 누적 변수 초기화와 똑같은 함정이에요.

패턴 빈 리스트 + for + if + append = 필터링 오늘의 결론

정의 조건에 맞는 값만 골라 새 리스트에 담습니다. 네 줄이 곧 데이터 필터링입니다.

```

temps = [25, 32, 28, 35, 27, 31, 24, 33]
hot = []
for t in temps:
    if t > 30:
        hot.append(t)
print(hot)
print(len(hot))

```

```

▶ [32, 35, 31, 33]
▶ 4

```

왜 쓰나 출력만 하면 그 자리에서 사라지지만, 리스트에 담으면 **나중에 다시 쓸 수 있습니다** — 개수를 세고, 평균을 내고, 정렬하고, 파일로 저장하죠.

응용 조건이 두 갈래면 리스트를 두 개 준비해 나눠 담습니다.

```

temps = [1, 5, 2, 7, 4, 8, 10, 3]
high = []
low = []
for t in temps:
    if t < 5:
        low.append(t)
    else:
        high.append(t)
print("high:", high)
print("low:", low)

```

```

▶ high: [5, 7, 8, 10]
▶ low: [1, 2, 4, 3]

```

데이터 처리 4패턴 — 그릇 준비 → for → (if) → 담기

패턴	뼈대	결과
출력	for → print	화면에만 (사라짐)
조건 선별	for + if → print	조건 맞는 값만 화면에
계산	0에서 시작 → total +=	합계 · 평균 (숫자 하나)
새 리스트	[] → append	재사용 가능한 리스트

STEP 7

리스트 안의 리스트

표처럼 행과 열로

리스트 중첩 리스트 리스트가 리스트를 담는다

정의 리스트의 값으로 또 리스트를 넣은 구조입니다. 표(행·열)처럼 한 줄에 여러 값이 묶인 데이터를 담습니다.

```
rows = [["펌프", 25], ["모터", 32], ["압축기", 28]]
print(rows[0])          # ['펌프', 25] ← 행 하나
print(type(rows))       # <class 'list'>
print(type(rows[0]))    # <class 'list'> ← 안쪽도 리스트
print(rows[1][1])       # 32 ← 대괄호 두 번
```

```
▶ ['펌프', 25]
▶ <class 'list'>
▶ <class 'list'>
▶ 32
```

왜 쓰나 바깥 대괄호가 **행**, 안쪽이 **열**입니다. CSV 한 줄을 그대로 담는 구조라, 앞으로 다룰 표 데이터의 기본형이에요.

응용 rows[1][1]이 어떻게 32가 되는지 한 단계씩 풀면 이렇습니다.

```
# 1. rows[1]          → ["모터", 32]
# 2. ["모터", 32][1] → 앞 리스트의 1번 인덱스
# 3. 결과             → 32
```



```
# 반복으로 전체 훑기
for row in rows:
    print(row[0], "온도", row[1])
```

```
▶ 펌프 온도 25
▶ 모터 온도 32
▶ 압축기 온도 28
```

개념

rows는 리스트를 담은 큰 리스트, row는 그 안의 작은 리스트 하나(예: ["펌프", 25])입니다. 이름을 단수/복수로 구분하면 헷갈리지 않아요.

패턴 중첩 리스트에서 한 열만 뽑기 표에서 특정 항목만

정의 반복하며 각 행의 같은 위치 값만 모으면, 표에서 한 열을 뽑아낸 것과 같습니다.

```
rows = [{"펌프", 25}, {"모터", 32}, {"압축기", 28}]

names = []
temps = []
for row in rows:
    names.append(row[0])    # 0번 열 = 이름
    temps.append(row[1])    # 1번 열 = 온도

print(names)
print(temps)
print("평균 온도:", sum(temps) / len(temps))
```

▶ ['펌프', '모터', '압축기']
▶ [25, 32, 28]
▶ 평균 온도: 28.333333333333332

왜 쓰나 표 형태 데이터에서 "온도 열만 뽑아 평균 내기" 같은 작업이 이 패턴입니다. STEP 6의 새 리스트 만들기를 중첩 리스트에 적용한 것이죠.

응용 조건까지 붙이면 "기준을 넘는 행"만 골라낼 수 있습니다. 행 전체를 담으면 이름과 값이 짝을 유지합니다.

```
rows = [{"펌프", 25}, {"모터", 32}, {"압축기", 28}]

hot_rows = []
for row in rows:
    if row[1] > 27:          # 온도 열로 판정
        hot_rows.append(row) # 행 전체를 담기

for name, temp in hot_rows: # 행을 두 변수로 나눠 받기
    print(name, temp, "고온")
```

▶ 모터 32 고온
▶ 압축기 28 고온

설비 적용

CSV 한 줄이 곧 한 행입니다. 5일치의 `split(",")` 결과를 리스트에 모으면 정확히 이 구조가 되어, 파일에서 읽은 데이터를 그대로 처리할 수 있습니다.

STEP 8

튜플 — 묶어서 보호하기

소괄호와 심표의 규칙

센서 이름과 값을 따로 두면 순서가 어긋나는 순간 짝이 깨집니다. 관련된 데이터는 처음부터 하나로 묶어두는 것이 안전한데, 그 도구가 튜플입니다.

튜플 튜플 만들기 (,) 소괄호 + 심표

정의 소괄호 안에 값을 심표로 구분해 묶습니다. 리스트가 대괄호였다면 튜플은 소괄호입니다.

```
sensor = ("모터온도", 78)
print(sensor)           # ('모터온도', 78)
print(type(sensor))     # <class 'tuple'>

sensor = "모터온도", 78  # 괄호 없어도 심표만 있으면 튜플
print(type(sensor))     # <class 'tuple'>
```

```
▶ ('모터온도', 78)
▶ <class 'tuple'>
▶ <class 'tuple'>
```

왜 쓰나 좌표(x, y)·연월일처럼 **짜지어 다녀야 하는 값**을 묶습니다. 값이 흩어져 순서가 틀어지는 사고를 원천 차단하죠.

자주 하는 실수

값이 **하나뿐인 튜플**은 뒤에 심표가 꼭 필요합니다. (78)은 그냥 숫자 78이고(괄호를 계산용으로 해석), (78,) 이라야 튜플입니다.

표기	타입	설명
("모터온도", 78)	tuple	괄호 0 · 값 2개 — 정상
"모터온도", 78	tuple	괄호 없어도 심표가 튜플을 만듦
(78,)	tuple	단일값 — 심표 필수
(78)	int	계산용 괄호로 해석됨
()	tuple	빈 튜플
{"모터온도", 78}	set	중괄호는 튜플이 아님 — 셋

자주 하는 실수

수업 코드의 주석에 "마지막 값에는 꼭 쉼표를 붙여야 튜플로 인식한다"고 적혀 있는데, 이건 **단일값 튜플에만** 해당합니다. ("모터온도", 78)은 쉼표가 없어도 튜플이에요(코드에서 직접 확인됩니다). 정확히는 **"값이 하나면 쉼표 필수, 둘 이상이면 선택"**입니다.

튜플 인덱싱·슬라이싱 리스트와 동일

정의 대괄호에 번호를 넣어 값을 꺼내고, 콜론으로 구간을 자릅니다. 규칙은 리스트·문자열과 완전히 같습니다.

```
sensor = ("모터온도", 78, "C")
print(sensor[0])          # 모터온도
print(sensor[-1])         # C   (길이를 몰라도 맨 끝)

s = ("strymon", "bigskymx", "timeline")
print(s[1][0:6])          # bigsky ← 튜플에서 꺼낸 문자열을 다시 슬라이싱
```

- ▶ 모터온도
- ▶ C
- ▶ bigsky

왜 쓰나 0부터 시작, 음수는 뒤에서, 끝 번호 미포함 — 6일차·5일차에 배운 규칙이 그대로 적용됩니다. 자료형이 달라도 접근 문법은 통일돼 있어요.

응용 s[1][0:6]처럼 대괄호를 이어 쓰면 "꺼낸 값에 다시 접근"이 됩니다. 중첩 리스트의 rows[1][1]과 같은 원리예요.

튜플 불변성 — 바꿀 수 없다 튜플의 정체성

정의 튜플은 한 번 만들면 값을 바꾸거나 추가·삭제할 수 없습니다. 이 성질을 **불변(immutable)**이라 합니다.

```
sensor = ("모터온도", 78)
sensor[1] = 80
# TypeError: 'tuple' object does not support item assignment
```

- ▶ TypeError — 튜플은 값을 바꿀 수 없음

왜 쓰나 버그가 아니라 **일부러 막아둔 안전장치**입니다. 리스트가 연필 글씨라면 튜플은 볼펜 글씨죠. 실수로 건드리면 그 자리에서 오류로 막아주니, 바뀌면 안 되는 데이터에 씁니다.

개념

선택 기준: 계속 추가·수정할 데이터는 리스트, 한 번 정해지면 안 바뀔 데이터는 튜플. 실무에선 센서 하나를 튜플로 묶고 그 튜플들을 리스트에 담아 씁니다.

개념 왜 바꿀 수 없는 튜플을 쓰는가 제약이 아니라 보호

정의 "못 바꾼다"는 것은 불편이 아니라, 실수로 데이터가 망가지지 않게 막아주는 안전장치입니다.

```
# ① 안전장치 — 실수로 건드릴 수 없음
설비ID = ("EQP-001", "1라인") # 한 번 정해지면 안 바뀔 값

# ② 즉시 차단 — 잘못 건드리면 그 자리에서 오류
# 설비ID[0] = "EQP-002" → TypeError로 바로 막힘

# ③ 리스트였다면 — 조용히 바뀌어 나중에 발견
설비ID_리스트 = ["EQP-001", "1라인"]
설비ID_리스트[0] = "EQP-002" # 오류 없이 그냥 바뀜
```

왜 쓰나 리스트라면 실수로 값을 덮어써도 오류가 안 나서 한참 뒤에야 발견합니다. 튜플은 그 순간 오류로 막아주죠.
볼펜으로 쓴 영수증이 연필 영수증보다 믿을 만한 것과 같은 이치입니다.

개념

덧붙여 튜플은 바뀌지 않기 때문에 나중에 배울 딕셔너리의 "키"로도 쓸 수 있습니다. 리스트는 바뀔 수 있어서 키가 될 수 없어요.

치트시트

오늘 배운 것 한 장 정리

A-1. 반복과 흐름 제어

형태	하는 일	언제
<code>for i in range(n)</code>	n번 반복	횟수를 아는 경우
<code>while 조건:</code>	조건이 참인 동안	횟수를 모르는 경우
<code>while True: ... break</code>	무한 반복 + 신호로 탈출	사용자 입력 처리
<code>break</code>	반복 전체 종료	찾았으니 그만
<code>continue</code>	이번 회차 건너뛴	거르고 계속

A-2. 데이터 처리 패턴

목적	빠대	주의
필터링	<code>[] → for → if → append</code>	빈 리스트는 반복 밖에서
가공	<code>[] → for → append(계산)</code>	원본은 그대로 보존
부분 평균	<code>total·count → total/count</code>	count 0이면 오류
최댓값	첫 값 → <code>if</code> 크면 갱신	내장 <code>max()</code> 와 대조
존재 확인	<code>found=False → True + break</code>	반복 뒤에 판단

A-3. 자료형 선택

자료형	괄호	값 변경	쓰는 곳
리스트	<code>[]</code>	가능	계속 추가·수정할 데이터
튜플	<code>(,)</code>	불가능	한 번 정해지면 안 바뀔 짝

부록 B

오류·함정 사전

오늘 코드에서 걸려 넘어지기 쉬운 곳

X 이렇게 하면	무슨 일	✓ 바로잡기
<code>while</code> 에 갱신 없음	무한 루프	<code>count += 1</code> 추가 · Ctrl+C
반복 안에서 값 되돌림	무한 루프	초기화는 반복 밖
시작값이 이미 조건 만족	0회 실행	범위 밖 값으로 시작
<code>total / count (count=0)</code>	ZeroDivisionError	<code>if count > 0:</code>
빈 리스트를 <code>for</code> 안에	매번 초기화	반복 밖에서 선언
<code>(78)</code> 단일값 튜플	그냥 int	<code>(78,)</code>
<code>{"a", 78}</code> 튜플로 착각	set이 됨	<code>("a", 78)</code>
<code>print(lst.sort())</code>	None 출력	정렬 후 다음 줄에 출력

B-2. 수업 코드에서 짚을 점

```
# ① 주석이 실제와 다름 — "마지막 값에 꼭 쉼표" 는 단일값에만 해당
sensor = ("모터온도", 78)      # 끝 쉼표 없어도 tuple ← 코드가 증명
sensor = (78,)                # 단일값일 때만 쉼표 필수

# ② 중괄호는 튜플이 아니라 셋
sensor = {"모터온도", 78}     # <class 'set'> - 튜플 아님

# ③ 불리언 비교는 생략이 관례
while found != True:          # 동작은 하지만
while not found:              # ✓ 더 파이썬다운 표현
```

개념

`print(low.sort())`가 None을 출력한다는 것을 코드 주석에서 스스로 짚어두신 부분은 정확합니다. 6일차에 배운 "리스트 메서드는 원본을 바꾸고 None을 반환한다"가 그대로 적용된 사례예요.

종합 실습 & 다음 시간

오늘의 도구를 한 흐름으로

실습 6은 오늘 배운 것이 전부 들어간 종합 문제입니다. 전체 평균을 구하고, 고온만 걸러 새 리스트를 만들고, 그 부분 평균까지 비교합니다.

```
temps = [25, 32, 28, 35, 27, 31, 24, 33, 29, 36]
```

```
# ① 전체 평균 — 누적 패턴
```

```
total = 0
for t in temps:
    total += t
print("전체 평균:", total / len(temps))
```

```
# ② 고온만 새 리스트로 — 필터링 패턴
```

```
hot = []
for t in temps:
    if t > 30:
        hot.append(t)
```

```
# ③ 부분 평균 — 조건에 맞는 개수로 나눔
```

```
print("고온 개수:", len(hot))
print("고온 평균:", sum(hot) / len(hot))
```

▶ 전체 평균: 30.0

▶ 고온 개수: 5

▶ 고온 평균: 33.4

설비 적용

전체 평균 30.0과 고온 평균 33.4를 나란히 보면, "평균은 정상인데 일부 구간이 위험하다"는 판단이 가능해집니다. 전체 평균만 보면 놓치는 신호를 부분 평균이 잡아내는 것 — 이것이 예지보전에서 구간 분석을 하는 이유입니다.

다음 시간 예고

다음 주제	한 줄 미리보기
튜플 언패킹	<code>name, value = sensor</code> · 반복문에서 <code>for n, v in sensors</code>

다음 주제	한 줄 미리보기
튜플 리스트	센서 하나를 튜플로 묶고 리스트에 모으는 실무 기본형
셋(set)과 집합 연산	중복 제거 · 합집합 · 교집합 · 차집합으로 두 목록 비교
딕셔너리	이름으로 값을 찾는 자료형 — 묶는 튜플, 거르는 셋, 찾는 딕셔너리

연결

교안 06_01은 튜플(Part 1)과 셋(Part 2)으로 되어 있는데, 오늘은 **튜플 만들기**까지만 나갔습니다. 언패킹 · 튜플 리스트 · 정렬과 셋 전체는 다음 시간이라 이 책에서는 예고로만 담았어요. 특히 셋의 집합 연산은 나중에 pandas로 두 표를 비교할 때 그대로 쓰입니다.

오늘의 성취와 다음 한 걸음

반복이 데이터 처리가 되는 날

오늘로 반복문이 완성되었습니다. `for`로 정해진 횟수를, `while`로 조건이 만족될 때까지 돌 수 있게 됐고, `break`·`continue`로 도중에 흐름을 바꾸는 법까지 익혔습니다. 그리고 그 반복을 리스트에 적용해 조건에 맞는 값만 골라 새 리스트로 만드는 — 데이터 분석의 가장 기본적인 동작을 직접 손으로 구현했습니다.

개념

복습 루틴 추천: ① `while`을 쓸 때마다 시작값·종료 조건·갱신 셋을 손가락으로 짚어 확인 → ② `break` 있는 검색과 없는 개수 세기를 각각 따보며 차이 체감 → ③ `[]` → `for` → `if` → `append` 네 줄을 눈 감고 쓸 수 있을 때까지 반복 → ④ 튜플은 `(78)`과 `(78,)`의 타입을 직접 찍어 확인. 특히 "**필터링 네 줄**"은 앞으로 매일 쓰게 될 패턴이라 몸에 붙여두면 좋습니다.